

Hyperparameter Tuning using Optuna | Bayesian Optimization (TPE) with Optuna

1 Introduction

- **Hyperparameter tuning** is the process of finding the best set of parameters for a model to maximize its performance.
- Manual search or simple grid/random search is often inefficient, especially with many parameters.
- **Optuna** is a modern framework for **automatic hyperparameter optimization** using **Bayesian Optimization** (specifically the **Tree-structured Parzen Estimator – TPE** algorithm).

 Optuna Research Paper: <https://arxiv.org/pdf/1907.10902>

 TPE / Bayesian Optimization Paper: <https://arxiv.org/pdf/2304.11127>

2 Why Use Optuna?

- **Efficient:** Uses Bayesian optimization (TPE) to intelligently sample promising hyperparameters.
 - **Pruning:** Stops unpromising trials early → saves time.
 - **Flexible:** Works with any ML/DL library (scikit-learn, XGBoost, LightGBM, PyTorch, TensorFlow, etc.).
 - **Visualizations:** Built-in plots for optimization history, parameter importance, and more.
-

3 Core Concepts

- **Study:** An optimization session (defines objective, search space, and sampler).
 - **Trial:** A single evaluation of hyperparameters.
 - **Sampler:** Strategy for suggesting parameters (TPE is default).
 - **Pruner:** Stops trials early if results are poor.
-

4 Bayesian Optimization (TPE)

- Instead of random guessing, TPE models:
 1. $l(x)$: Distribution of good parameter configurations.
 2. $g(x)$: Distribution of bad configurations.
-

- Chooses new parameters that **maximize the ratio $l(x)/g(x)$** → focuses on promising regions of the search space.
 - Leads to faster convergence to optimal hyperparameters.
-

5 Steps to Use Optuna

1. **Install:**
2. pip install optuna
3. **Define Objective Function:**
 1. Receives a trial object.
 2. Suggests parameters via trial.suggest_*() (e.g., suggest_float, suggest_int).
 3. Returns a metric to minimize or maximize (e.g., validation loss).
4. **Create & Run Study:**

```
import optuna
```

```
def objective(trial):
```

```
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
```

```
    n_estimators = trial.suggest_int("n_estimators", 50, 500)
```

```
    model = XGBClassifier(learning_rate=lr, n_estimators=n_estimators)
```

```
    model.fit(X_train, y_train)
```

```
    preds = model.predict(X_valid)
```

```
    return 1 - accuracy_score(y_valid, preds) # minimize error
```

```
study = optuna.create_study(direction="minimize")
```

```
study.optimize(objective, n_trials=50)
```

Inspect Results:

```
print(study.best_trial.params)
```

```
study.best_value
```

6 Advanced Features

- **Pruning:**
 - ❖ Use built-in pruners (MedianPruner, SuccessiveHalvingPruner) to cut off bad trials.
- **Parallelization:**
 - ❖ Run trials on multiple CPUs/GPUs for speed.

- **Visual Analysis:**

```
optuna.visualization.plot_optimization_history(study)
```

```
optuna.visualization.plot_param_importances(study)
```

7 Key Insights

- Bayesian (TPE) optimization focuses exploration on promising hyperparameter regions.
 - Optuna's pruning makes it practical even for expensive models (e.g., deep learning).
 - For reproducibility, set `sampler=TPESampler(seed=42)`.
-

Quick Recap

Optuna + TPE = smart, efficient hyperparameter search.

It models good vs. bad configurations and chooses the next set of parameters that are most likely to improve performance.
